- Authentication in ASP.NET Core is provided by `AuthenticationMiddleware` and a number of authentication services. These services are responsible for setting the current principal when a user logs in, saving it to a cookie, and loading the principal from the cookie on subsequent requests.
- The `AuthenticationMiddleware` is added automatically by `WebApplication`. You can ensure that it's inserted at a specific point in the middleware pipeline by calling `UseAuthentication()`. It must be placed before any middleware that requires authentication, such as `UseAuthorization()`.
- ASP.NET Core Identity handles low-level services needed for storing users in a database, ensuring that their passwords are stored safely, and for logging users in and out. You must provide the UI for the functionality yourself and wire it up to the Identity subsystem.
- The Microsoft.AspNetCore.Identity.UI package provides a default UI for the Identity system and includes email confirmation, MFA, and external login provider support. You need to do some additional configuration to enable these features.
- The default template for Web Application with Individual Account Authentication uses ASP.NET Core Identity to store users in the database with EF Core. It includes all the boilerplate code required to wire the UI up to the Identity system.
- You can use the `UserManager<T>` class to create new user accounts, load them from the database, and change their passwords. `SignInManager<T>` is used to sign a user in and out by assigning the principal for the request and by setting an authentication cookie. The default UI uses these classes for you, to facilitate user registration and login.